

---

# Projeto Computacional de Otimização de Algoritmos

---

*Alunos*

Beatriz Coelho 107113  
Gonçalo Duarte 116131  
Liane Lin 103773  
Tomás Orfão 106872

*Professores*

João Freitas Xavier

21 out, 2025

## Índice

|     |                                                                               |    |
|-----|-------------------------------------------------------------------------------|----|
| 1   | Task 1: Deterministic Target Numerical Solution (Appendix A.1)                | 1  |
| 2   | Task 2: Theoretical Proof of Tracking Error vs. Control Effort                | 2  |
| 3   | Task 3: Uniqueness of Solution                                                | 3  |
| 4   | Task 4: Random Target with No Information Numerical Solution (Appendix A.2)   | 5  |
| 5   | Task 5: Flaw in Incomplete Formulation (5)                                    | 6  |
| 6   | Task 6: Extra Constraints for Problem (5)                                     | 6  |
| 7   | Task 7: Random Target with Midway Information Numerical Solution              | 7  |
| 8   | Task 8: Non-convexity of Circle Fitting Problem                               | 8  |
| 9   | Task 9: Least-Squares Approach with Non-Uniform Dataset (Appendix A.4)        | 9  |
| 10  | Task 10: Levenberg-Marquardt Approach with Non-Uniform Dataset (Appendix A.5) | 10 |
| 11  | Task 11: Distributed Circle Fitting via ALM and BCD (Appendix A.6)            | 12 |
| A   | Code and Output for Numerical Tasks                                           | 13 |
| A.1 | Code and Output for Task 1 . . . . .                                          | 13 |
| A.2 | Code and Output for Task 4 . . . . .                                          | 17 |
| A.3 | Code and Output for Task 7 . . . . .                                          | 20 |
| A.4 | Code and Output for Task 9 . . . . .                                          | 24 |
| A.5 | Code and Output for Task 10 . . . . .                                         | 26 |
| A.6 | Code and Output for Task 11 . . . . .                                         | 29 |

# 1 Task 1: Deterministic Target Numerical Solution (Appendix A.1)

## Analytical formulation of the trade-off and Results

The observed relation between the tracking error ( $TE^*$ ) and the control effort ( $CE^*$ ) arises from the given optimization problem:

$$\min_{x, u} J_\rho(x, u) = TE(x) + \rho CE(u),$$

where

$$TE(x) = \sum_{t=1}^T \|Ex(t) - q(t)\|_2, \quad CE(u) = \sum_{t=1}^{T-1} \|u(t)\|_2^2,$$

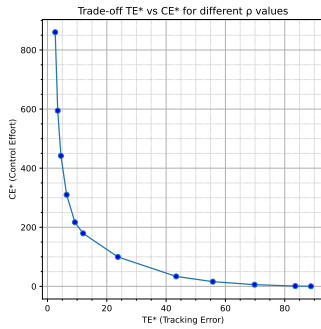
and the system dynamics are given by

$$x(t+1) = Ax(t) + Bu(t), \quad x(0) = x_{\text{init}}.$$

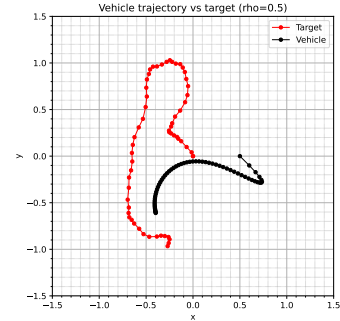
The parameter  $\rho > 0$  determines the trade-off between the two competing objectives:

- For large  $\rho$ , the cost penalizes the control effort more strongly. The optimal control inputs  $u(t)$  are smoother and smaller in magnitude, leading to a higher tracking error  $TE^*$ .
- For small  $\rho$ , the controller can apply larger control actions to better follow the target trajectory, which reduces  $TE^*$  but increases  $CE^*$ .

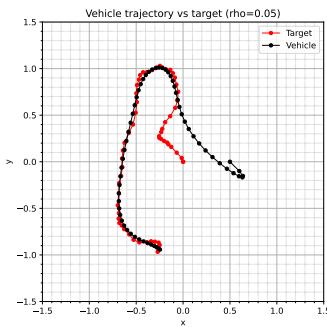
Thus, varying  $\rho$  generates a set of optimal pairs  $(TE_\rho^*, CE_\rho^*)$  that lie on a Pareto trade-off curve. This curve represents the fundamental balance between tracking precision and control energy expenditure. Here we present, the previously mentioned trade-off and some results for different  $\rho = 0.5, 0.05$  and  $0.005$ .



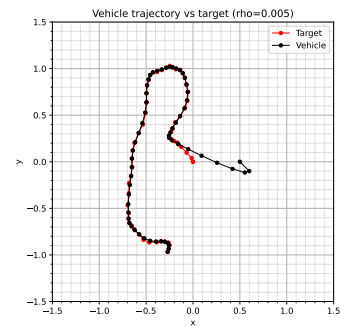
(a) Trade-off between  $TE^*$  and  $CE^*$ .



(b)  $\rho = 0.5$



(c)  $\rho = 0.05$



(d)  $\rho = 0.005$

Figure 1: Grid of some outputted plots

## 2 Task 2: Theoretical Proof of Tracking Error vs. Control Effort

### Relation between $TE_\rho^*$ and $CE_\rho^*$

*Proof of the trade-off property.* We aim to prove that if

$$TE_{\rho_a}^* \leq TE_{\rho_b}^*,$$

then necessarily

$$CE_{\rho_a}^* \geq CE_{\rho_b}^*.$$

This formalizes the intuitive trade-off: as the weight  $\rho$  on the control effort increases, the optimal control effort decreases while the tracking error increases.

Consider the cost function

$$J_\rho(x, u) = TE(x) + \rho CE(u),$$

and let  $(x_a^*, u_a^*)$  and  $(x_b^*, u_b^*)$  denote the optimal solutions for  $\rho_a$  and  $\rho_b$ , respectively. By definition,

$$TE_a^* = TE(x_a^*), \quad CE_a^* = CE(u_a^*), \quad TE_b^* = TE(x_b^*), \quad CE_b^* = CE(u_b^*).$$

From the optimality of  $(x_a^*, u_a^*)$  for  $J_{\rho_a}$ :

$$J_{\rho_a}(x_a^*, u_a^*) \leq J_{\rho_a}(x_b^*, u_b^*)$$

which implies

$$TE_a^* + \rho_a CE_a^* \leq TE_b^* + \rho_a CE_b^*.$$

Rearranging:

$$\rho_a(CE_a^* - CE_b^*) \leq TE_b^* - TE_a^*. \quad (1)$$

Similarly, from the optimality of  $(x_b^*, u_b^*)$  for  $J_{\rho_b}$ :

$$J_{\rho_b}(x_b^*, u_b^*) \leq J_{\rho_b}(x_a^*, u_a^*)$$

which yields

$$TE_b^* + \rho_b CE_b^* \leq TE_a^* + \rho_b CE_a^*,$$

or equivalently,

$$TE_b^* - TE_a^* \leq \rho_b(CE_a^* - CE_b^*). \quad (2)$$

Combining (1) and (2):

$$\rho_a(CE_a^* - CE_b^*) \leq TE_b^* - TE_a^* \leq \rho_b(CE_a^* - CE_b^*).$$

Given that  $TE_a^* \leq TE_b^*$ , the middle term is non-negative:

$$TE_b^* - TE_a^* \geq 0.$$

Hence, from the right inequality,

$$\rho_b(CE_a^* - CE_b^*) \geq 0.$$

Since  $\rho_b > 0$ , dividing both sides by  $\rho_b$  gives

$$CE_a^* - CE_b^* \geq 0,$$

that is,

$$\boxed{CE_a^* \geq CE_b^*}.$$

□

### 3 Task 3: Uniqueness of Solution

We are asked to show that the optimization problem (3) has a unique solution. We follow the hint to rewrite the problem as an unconstrained optimization problem that depends only on the stacked control vector  $U^T = (u(1), u(2), \dots, u(T-1))^T$ .

#### Rewriting the Problem

After computing  $x(t+1) = Ax(t) + Bu(t)$  with  $x(1) = x_{\text{init}}$  for the following  $x_i$ , we get:

$$\begin{cases} x(1) = x_{\text{init}}, \\ x(2) = Ax_{\text{init}} + Bu(1), \\ x(3) = A^2x_{\text{init}} + ABu(1) + Bu(2), \\ x(4) = A^3x_{\text{init}} + A^2Bu(1) + ABu(2) + Bu(3), \\ \vdots \end{cases}$$

As one can see, heuristically, this algorithm can be written in a closed form. At any time  $t$  and can be expressed as an affine function of  $U$ :

$$x(t) = A^{t-1}x_{\text{init}} + \sum_{k=1}^{t-1} A^{t-1-k}Bu(k)$$

The vehicle's position  $p(t) = Ex(t)$  is therefore also an affine function of  $U$ .

The cost function in (3) can now be written as a single unconstrained function  $F(U)$ :

$$F(U) = \underbrace{\sum_{t=1}^T \|Ex(t) - q(t)\|_2}_{g(U) \equiv TE} + \rho \underbrace{\sum_{t=1}^{T-1} \|u(t)\|_2^2}_{h(U) \equiv CE}$$

To prove a unique solution exists, we will show that  $F(U)$  is strictly convex.

#### The Tracking Error (TE) term $g(U)$

*Proof of convexity.* Consider

$$g(U) = \sum_{t=1}^T \|Ex(t) - q(t)\|_2.$$

- For each  $t$ ,  $Ex(t) - q(t)$  is an affine function of  $U$ .
- The function  $f(v) = \|v\|_2$  is convex.
- The composition of a convex function with an affine function is convex.

Hence, each term  $\|Ex(t) - q(t)\|_2$  is convex. Since the sum of convex functions is convex,

$g(U) \text{ is convex.}$

□

### The Control Effort (CE) term $h(U)$

*Proof of strict convexity.* Consider

$$h(U) = \rho \sum_{t=1}^{T-1} \|u(t)\|_2^2 = \rho \|U\|_2^2, \quad \text{with } \rho > 0.$$

- The squared L2-norm  $f(U) = \|U\|_2^2$  is **strictly convex**.
- Multiplying a strictly convex function by a positive scalar preserves strict convexity.

Hence,

$h(U)$  is strictly convex.

□

### The Total Cost $F(U)$

*Proof of strict convexity.* Consider

$$F(U) = g(U) + h(U),$$

where  $g(U)$  is convex and  $h(U)$  is strictly convex.

- According to the convexity pattern, the sum of a convex with a strictly convex function is **strictly convex**.

Therefore,

$F(U)$  is strictly convex.

□

### Conclusion

*Proof of uniqueness of the solution.* A strictly convex function has at most one global minimum.

Since  $F(U)$  is continuous and coercive (because the term  $\rho \|U\|_2^2$  with  $\rho > 0$  ensures  $F(U) \rightarrow \infty$  as  $\|U\| \rightarrow \infty$ ), the minimum is attained.

Therefore, the optimization problem (3) admits a unique global minimum.

□

## 4 Task 4: Random Target with No Information Numerical Solution (Appendix A.2)

We solve problem (4) with CVXPY using  $\rho = 0.1$ . The dynamics and initial condition are those in the statement, and the cost is implemented exactly as

$$\min_{x,u} p_1 \sum_{t=1}^T \|Ex(t) - q_1(t)\|_2^2 + p_2 \sum_{t=1}^T \|Ex(t) - q_2(t)\|_2^2 + \rho \sum_{t=1}^{T-1} \|u(t)\|_2^2,$$

$$x(1) = x_{\text{initial}}, \quad x(t+1) = Ax(t) + Bu(t).$$

For each instance we compute the optimal  $(x^*, u^*)$  and plot the two target trajectories  $q_1$  (red) and  $q_2$  (blue) together with the single vehicle trajectory (black). The behaviour matches the model: since there is no information about which target is true, the vehicle chooses one path that minimizes the *expected* tracking error  $p_1 \text{TE}_1 + p_2 \text{TE}_2$  plus effort. As  $p_1$  grows (and  $p_2$  shrinks), the vehicle bends toward  $q_1$ ; when  $p_2$  is larger, it bends toward  $q_2$ . In the balanced case  $(0.5, 0.5)$ , the vehicle stays roughly between the two, trading small errors to each rather than committing to one. Where the targets diverge strongly, the vehicle follows a compromise path that reduces the weighted error over the horizon. The trajectories remain smooth because  $\rho = 0.1$  penalizes aggressive inputs, so the vehicle still lags slightly when a target turns sharply. Overall, the sequence of plots shows a continuous shift from tracking  $q_2$  to tracking  $q_1$  as the probabilities move from  $(0, 1)$  to  $(1, 0)$ , exactly as expected from the average-cost formulation.

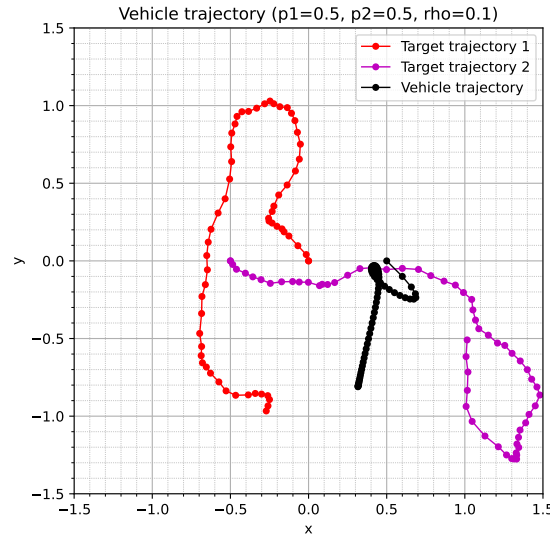


Figure 2: Vehicle trajectory for the balanced case  $(p_1, p_2) = (0.5, 0.5)$  and  $\rho = 0.1$ .

In this balanced case  $(p_1, p_2) = (0.5, 0.5)$ , the vehicle (black) follows a path between the two possible targets,  $q_1$  (red) and  $q_2$  (blue), minimizing the expected tracking error to both. This plot represents the typical behaviour observed in all simulations; the results for the other  $(p_1, p_2)$  combinations are included in the appendix.

## 5 Task 5: Flaw in Incomplete Formulation (5)

The formulation (5) models a situation where the target may follow one of two possible trajectories,  $q_1(t)$  or  $q_2(t)$ , with associated probabilities  $p_1$  and  $p_2$ . The optimization seeks to minimize the expected tracking error and control effort,

$$\min_{x_1, u_1, x_2, u_2} \sum_{k=1}^2 p_k \left( \sum_{t=1}^T \|Ex_k(t) - q_k(t)\|_2^2 + \rho \sum_{t=1}^{T-1} \|u_k(t)\|_2^2 \right)$$

subject to the system dynamics

$$x_k(t+1) = Ax_k(t) + Bu_k(t), \quad x_k(1) = x_{\text{initial}}.$$

In this situation, the vehicle does not initially know which path the target will take. The correct trajectory is only revealed at time  $t = 25$ . Until that moment ( $t \leq 24$ ), the vehicle has exactly the same information in both scenarios, so it must apply the same control actions and follow the same trajectory. However, formulation (5) does not include any connection between  $(x_1, u_1)$  and  $(x_2, u_2)$ , which allows the optimizer to create two completely independent trajectories from the start — as if there were two different vehicles acting separately.

To properly represent a single vehicle that operates under uncertainty, it is necessary to make both systems identical up to the information time. This means that the control applied before  $t = 25$  cannot depend on which target will later be revealed, because decisions taken before an event cannot anticipate its outcome.

Without these additional constraints, formulation (5) would ignore the problem's timing and information limits, leading to unrealistic strategies that assume the vehicle already knows the target's path. By forcing the states and controls to remain equal until the information becomes available, the model correctly represents a single vehicle making optimal decisions without access to future information.

## 6 Task 6: Extra Constraints for Problem (5)

Considering the reasoning from the previous task, the correct formulation of Problem (5) must explicitly include the constraints that force the two trajectories and controls to coincide until the information time  $t = 25$ . The corrected optimization problem can therefore be written as

$$\begin{aligned} \min_{x_1, x_2, u_1, u_2} \quad & \sum_{k=1}^2 p_k \left( \sum_{t=1}^T \|Ex_k(t) - q_k(t)\|_2^2 + \rho \sum_{t=1}^{T-1} \|u_k(t)\|_2^2 \right) \\ \text{s.t.} \quad & x_k(t+1) = Ax_k(t) + Bu_k(t), \quad x_k(1) = x_{\text{initial}}, \quad k = 1, 2, \\ & x_1(t) = x_2(t), \quad t = 1, 2, \dots, 25, \\ & u_1(t) = u_2(t), \quad t = 1, 2, \dots, 24. \end{aligned}$$

The first set of constraints describes the system dynamics for each possible scenario, while the following ones ensure that both systems share the same states and control inputs before the information time. After  $t = 25$ , the two control sequences can diverge, representing the moment when the vehicle learns which target it must follow and adjusts its behaviour accordingly. This corrected formulation makes the optimization problem physically consistent and aligned with how information becomes available over time.

The model obtained here will be used in the next task to calculate and analyse the optimal trajectories and control actions.

## 7 Task 7: Random Target with Midway Information Numerical Solution

In this section, a new optimization problem was considered, in which the objective is to determine the optimal trajectory of a vehicle tasked with following a target whose path is initially uncertain. The problem can be formulated as:

$$\min_{x,u} p_1 \sum_{t=1}^T \|\mathbb{E}[x(t)] - q_1(t)\|_2^2 + p_2 \sum_{t=1}^T \|\mathbb{E}[x(t)] - q_2(t)\|_2^2 + \rho \sum_{t=1}^{T-1} \|u(t)\|_2^2,$$

where the first two terms represent, once again, the expected tracking errors ( $TE_1$  and  $TE_2$ ) weighted by the probabilities  $p_1$  and  $p_2$  of the target following trajectories  $q_1$  and  $q_2$ , respectively, and the third term penalizes the control effort with weight  $\rho$ .

Non-anticipativity constraints enforce that the control inputs are identical for both scenarios up to the reveal time  $t_r = 25$ :

$$x_1(t) = x_2(t), \forall t \leq 25,$$

$$u_1(t) = u_2(t), \forall t \leq 24,$$

This, as seen previously, ensures that the vehicle does not act on information about the target trajectory before it becomes available. The solution yields two optimal vehicle trajectories,  $x_1$  and  $x_2$ , which coincide up to  $t = 25$ , representing an optimal compromise that minimizes the expected tracking error weighted by the probabilities. After  $t = 25$ , the trajectories diverge according to the observed target path:  $x_1$  tracks  $q_1$  and  $x_2$  tracks  $q_2$ . This formulation balances anticipatory positioning and adaptive response, simultaneously minimizing expected tracking error and control effort.

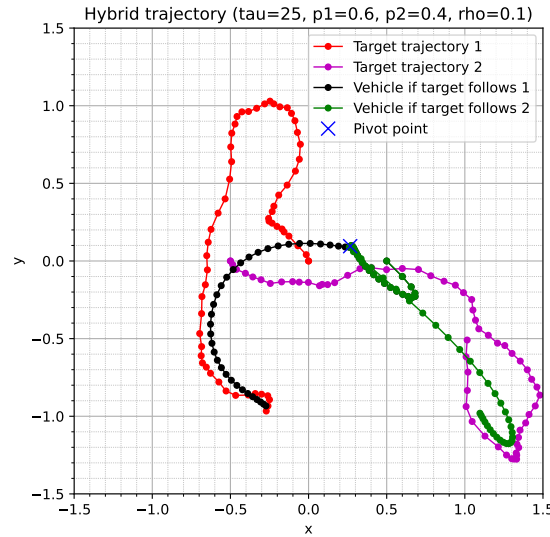


Figure 3: The two known possible trajectories that the target can execute, trajectory 1 (red) and trajectory 2 (magenta), and the two trajectories of our vehicle (black and green) corresponding to the control signals optimized for  $\rho = 0.1$ ,  $p_1 = 0.6$  and  $p_2 = 0.4$ . The trajectories  $x_1$  and  $x_2$  split at  $t = 25$ .

## 8 Task 8: Non-convexity of Circle Fitting Problem

We aim to show that the circle fitting problem is **non-convex** by demonstrating that its cost function is non-convex. The function is defined as:

$$f(c, R) = \sum_{n=1}^N (\|c - x_n\|_2 - R)^2$$

### Method

To prove that  $f(c, R)$  is non-convex, we will find a line on which its restriction is non-convex. We define this line as an affine function  $g(t) = p + td$  and form the composed function  $\Phi(t) = f(g(t))$ . If  $f$  were convex, then  $\Phi(t)$  would have to be convex. By finding a single case where  $\Phi(t)$  is not convex, we can prove that  $f$  is non-convex.

### Counterexample Construction

We first consider the simplified case of a single data point,  $N = 1$ , located at the origin,  $x_1 = (0, 0)$ . The cost function becomes:

$$f(c, R) = (\|c\|_2 - R)^2.$$

The domain of this function is  $\mathbb{R}^3$ , for the variable  $z = (c_1, c_2, R)$ . We define an affine path  $g(t)$  that fixes the radius at  $R = 1$  and moves the center along the  $c_1$ -axis:

$$g(t) = ((t, 0), 1)$$

The restriction of  $f$  to this path is the composed function  $\Phi(t) = f(g(t))$ :

$$\Phi(t) = f((t, 0), 1) = (\|(t, 0)\|_2 - 1)^2 = (|t| - 1)^2$$

### Proof of Non-Convexity

A function  $f$  is convex if and only if its restriction  $\Phi(t) = f(g(t))$  along any affine path  $g(t)$  is also convex.

The graph of restricted function  $\Phi(t) = (|t| - 1)^2$  plotted in Figure 4.

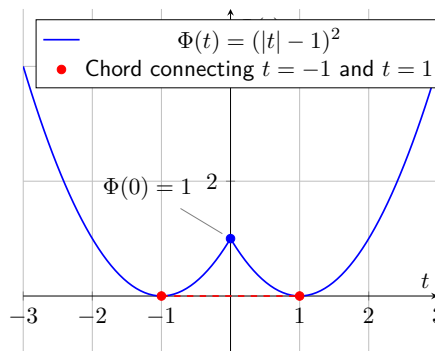


Figure 4: Graph of the non-convex restricted function  $\Phi(t) = (|t| - 1)^2$ .

The plot clearly shows that  $\Phi(t)$  is **non-convex**. It exhibits a "W" shape with two distinct global minima at  $t = -1$  and  $t = 1$ , rather than a single convex bowl. Furthermore, the function's value at the midpoint  $t = 0$  (where  $\Phi(0) = 1$ ) lies strictly above the chord connecting the points at  $t = -1$  and  $t = 1$  (where  $\Phi(-1) = \Phi(1) = 0$ ).

Since we have found an affine path  $g(t)$  for which the restriction  $\Phi(t)$  is non-convex, the original cost function  $f(c, R)$  is proven to be **non-convex**.

## 9 Task 9: Least-Squares Approach with Non-Uniform Dataset (Appendix A.4)

After loading the dataset from the file `circle_data_2.npy` and applying the LS approach, we got the following  $c_{ls}^*$  and  $R_{ls}^*$ :

$$c_{ls}^* = (-0.45, 1.50), \quad R_{ls}^* = 0.59$$

After obtaining these values, we were able to plot the circle with center  $c_{ls}^*$  and radius  $R_{ls}^*$ , giving us the following graph:

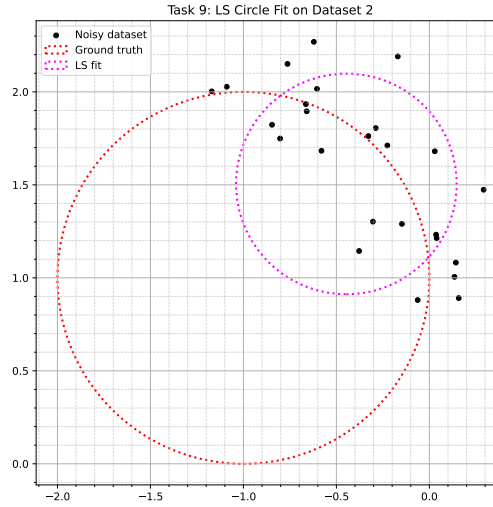


Figure 5: The ground truth circle (red) and the circle fitted by the LS approach (magenta).

The discrepancy between the circle obtained using the LS (Least Squares) approach and the ground truth circle arises because the LS method relies on a linearized approximation of an inherently nonlinear problem. In the presence of noise, the data points no longer lie exactly on the true circle, which makes the original nonlinear relation

$$\|x_n - c\|^2 = R^2$$

inconsistent across all samples. The LS approach reformulates this into a linear system and determines the circle parameters that minimize the overall algebraic error across all data points.

## 10 Task 10: Levenberg-Marquardt Approach with Non-Uniform Dataset (Appendix A.5)

For this task, we had to apply our implementation of the LM algorithm to the dataset from the file `circle_data_2.npy` from which we got the following  $\mathbf{c}_{lm}^*$  and  $R_{lm}^*$ :

$$\mathbf{c}_{lm}^* = (-0.88, 1.10), \quad R_{lm}^* = 0.91$$

After obtaining these values, we were able to plot the circle with center  $\mathbf{c}_{lm}^*$  and radius  $R_{lm}^*$ , giving us the following graph:

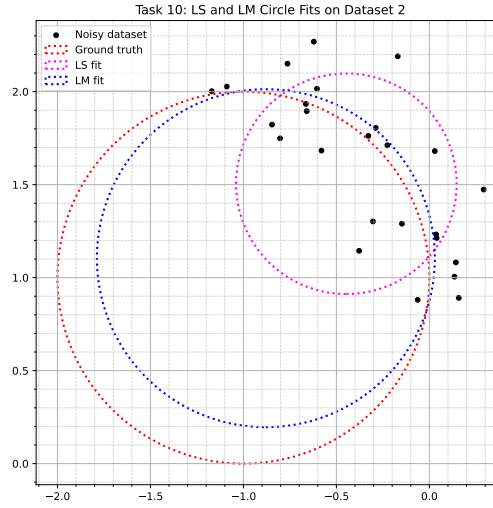


Figure 6: The ground truth circle (red), the circle fitted by the LS approach (magenta), and the circle fitted by the LM algorithm (blue).

Finally, we plotted the norm of the gradient of  $\mathbf{f}$  across iterations (35 iterations in our case) of the LM algorithm:

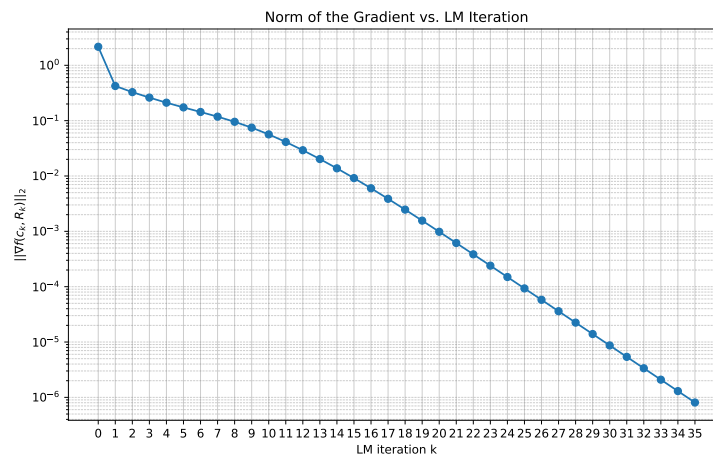


Figure 7: Norm of the gradient of the cost function  $\mathbf{f}$  across iterations of the LM algorithm.

The results clearly show the LM algorithm provides a much more accurate fit to the ground truth than the LS approach. This improvement is expected, as the LS method is a sub-optimal approach

that solves a simplified, linear approximation of the problem. The LM algorithm, in contrast, is an optimal method designed to directly minimize the true nonlinear cost function  $f(c, R)$ , which explicitly minimizes the squared geometric distances from the data points to the circle. The convergence plot confirms that the LM algorithm successfully found a local minimum where the gradient of this true cost function is near zero.

## 11 Task 11: Distributed Circle Fitting via ALM and BCD (Appendix A.6)

In task 11, we were asked to solve the following problem:

$$\min_{x_0, x_1, \dots, x_P} \sum_{p=1}^P \|A_p x_p - b_p\|_2^2$$

using the described ALM and BCD algorithms. In doing this, we were able to obtain both the center and the radius of the fitted circle:

$$c_{FL}^* = (1.22, 5.05), \quad R_{FL}^* = 4.08$$

in 4 ALM iterations, and with a corresponding number of BCD iterations per ALM iteration of 170, 3, 3 and 2 respectively. With this, we were able to plot the fitted circle by Federated Learning against the local datasets:

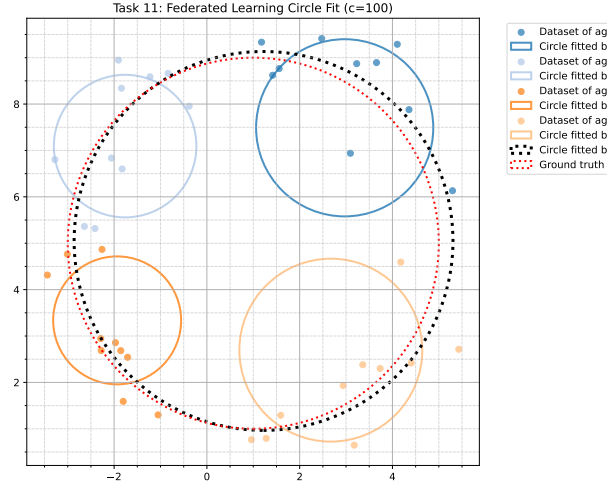


Figure 8: The four local datasets and the corresponding four local circles fitted by the four agents on their own, plus the circle fitted by Federated Learning.

Finally, we also plotted the maximum constraint violation across the iterations  $k$  of ALM:

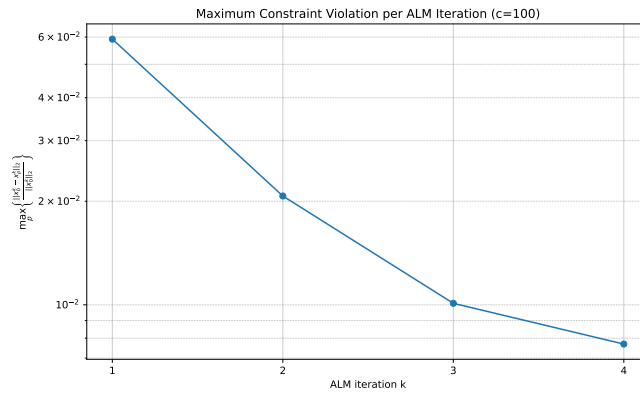


Figure 9: Maximum constraint violation across ALM iterations, for the case  $c = 100$ .

## A Code and Output for Numerical Tasks

### A.1 Code and Output for Task 1

#### Code

Listing 1: Python code for Task 1.

```
1 def optimize_vehicle_trajectory(A, B, E, x_init, q, T, lista_rho,
2   plot_trajectories=True, plot_tradeoff=True):
3
4   # Check if T does not exceed the target dimension
5   if T > q.shape[1]:
6       raise ValueError(f"T={T} is greater than the number of columns in target
7       q ({q.shape[1]})")
8
9   # Initialize lists
10  TE_list = []
11  CE_list = []
12  trajectories = []
13
14  # Loop over each rho
15  for rho in lista_rho:
16      # CVXPY variables
17      x_var = cp.Variable((4, T))
18      u_var = cp.Variable((2, T-1))
19
20      # System constraints
21      constraints = [x_var[:, 0] == x_init]
22      for t in range(T-1):
23          constraints.append(x_var[:, t+1] == A @ x_var[:, t] + B @ u_var[:, t]
24          ])
25
26      # TE term = sum of L2 norms
27      TE = cp.sum(cp.norm(E @ x_var - q[:, :T], axis=0))
28
29      # CE term = sum of squared L2 norms
30      CE = cp.sum_squares(u_var)
31
32      # Objective function
33      cost = TE + rho * CE
34
35      # Solve the optimization problem
36      prob = cp.Problem(cp.Minimize(cost), constraints)
37      prob.solve(solver=cp.SCS, verbose=False)
38
39      # Save metrics
40      TE_val = TE.value
41      CE_val = CE.value
42      TE_list.append(TE_val)
43      CE_list.append(CE_val)
44
45      # Save vehicle trajectory
46      trajectories.append(x_var.value.copy())
47
48  return trajectories, TE_list, CE_list
49
50 # Setting parameters
51 A = np.array([[1, 0, 0.1, 0],
52               [0, 1, 0, 0.1],
53               [0, 0, 0.8, 0],
54               [0, 0, 0, 0.8]])
```

```

53 B = np.array([[0, 0],
54               [0, 0],
55               [0.1, 0],
56               [0, 0.1]])
57
58 E = np.array([[1, 0, 0, 0],
59               [0, 1, 0, 0]])
60
61 x_init = np.array([0.5, 0, 1, -1])
62
63 # Load targets
64 q1 = np.load("target_1.npy")
65 q2 = np.load("target_2.npy")
66 q=q1
67
68 # Instance parameters
69 T = 60
70 p1, p2 = 0, 1
71
72 lista_rho = [10,5,2,1,0.5,0.2,0.1,0.05,0.02,0.01,0.005,0.002]
73
74 x_traj, TE_val, CE_val = optimize_vehicle_trajectory(A, B, E, x_init, q, T,
75               lista_rho, plot_trajectories=True, plot_tradeoff=True)
76
77 print("rho:\t (TE, CE)")
78 for rho, te, ce in zip(lista_rho, TE_val, CE_val):
79     print(f"{rho}\t ({te:.2f}, {ce:.2f})")
80
81 # Plot trade-off TE* vs CE*
82 plt.figure(figsize=(6,6))
83 plt.plot(TE_val, CE_val, 'o-', markerfacecolor='blue')
84 plt.xlabel("TE* (Tracking Error)")
85 plt.ylabel("CE* (Control Effort)")
86 plt.title("Trade-off TE* vs CE* for different values")
87 plt.minorticks_on()
88 plt.grid(which='major', linestyle='-', linewidth='0.8')
89 plt.grid(which='minor', linestyle=':', linewidth='0.5')
90 plt.savefig("export/task1/tradeoff_plot.pdf")
91 plt.show()
92
93 # Plot vehicle trajectories vs target
94 for i, rho in enumerate(lista_rho):
95     plt.figure(figsize=(6,6))
96     plt.plot(q[0,:T], q[1,:T], 'r-o', linewidth=1, ms=4, label='Target')
97     plt.plot(x_traj[i][0,:], x_traj[i][1,:], 'k-o', linewidth=1, ms=4, label='
Vehicle')
98     plt.xlabel('x')
99     plt.ylabel('y')
100     plt.title(f'Vehicle trajectory vs target (rho={rho})')
101     plt.legend()
102     plt.minorticks_on()
103     plt.grid(which='major', linestyle='-', linewidth='0.8')
104     plt.grid(which='minor', linestyle=':', linewidth='0.5')
105     plt.xlim(-1.5, 1.5)
106     plt.ylim(-1.5, 1.5)
107     plt.savefig(f"export/task1/trajectory_rho_{rho}.pdf")
108     plt.show()

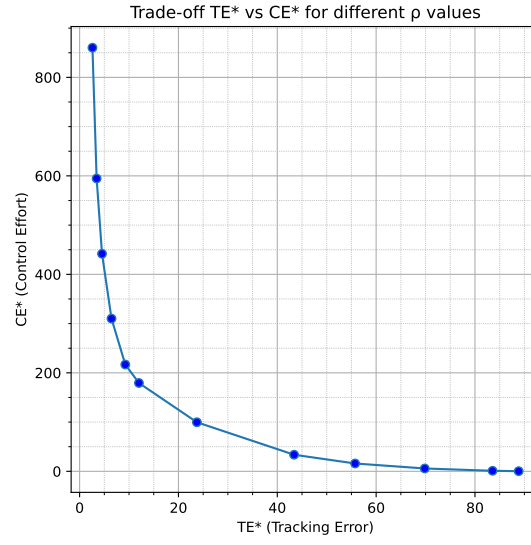
```

## Output

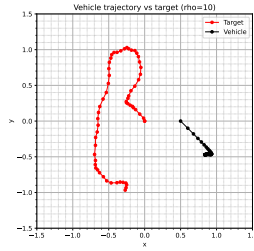
Listing 2: Console output for Task 1.

|       |                 |
|-------|-----------------|
| rho:  | (TE, CE)        |
| 10    | (88.84, 0.28)   |
| 5     | (83.56, 1.07)   |
| 2     | (69.84, 5.77)   |
| 1     | (55.75, 15.92)  |
| 0.5   | (43.40, 33.68)  |
| 0.2   | (23.73, 99.65)  |
| 0.1   | (12.00, 179.38) |
| 0.05  | (9.24, 216.86)  |
| 0.02  | (6.45, 310.13)  |
| 0.01  | (4.53, 441.79)  |
| 0.005 | (3.44, 594.65)  |
| 0.002 | (2.60, 860.26)  |

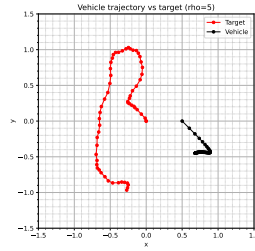
## Plots



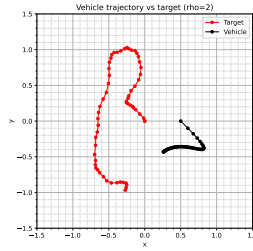
(a) Trade-off between TE\* and CE\*.



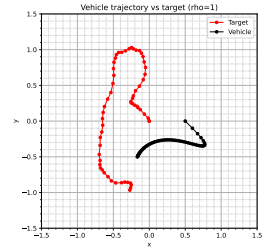
(b)  $\rho = 10$



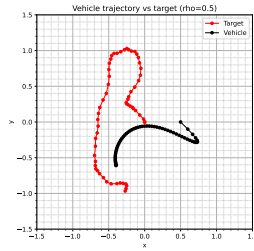
(c)  $\rho = 5$



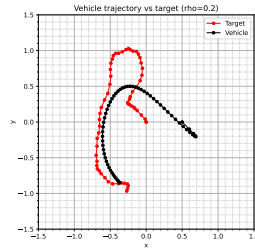
(d)  $\rho = 2$



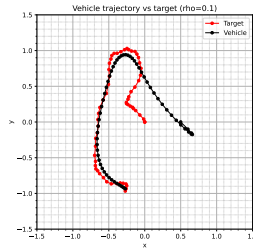
(e)  $\rho = 1$



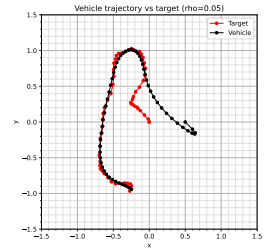
(f)  $\rho = 0.5$



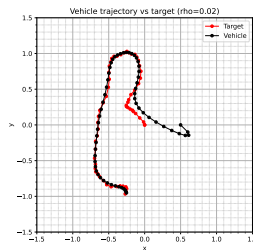
(g)  $\rho = 0.2$



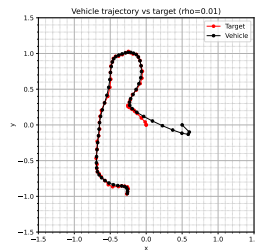
(h)  $\rho = 0.1$



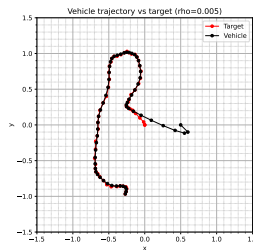
(i)  $\rho = 0.05$



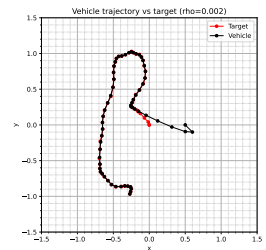
(j)  $\rho = 0.02$



(k)  $\rho = 0.01$



(l)  $\rho = 0.005$



(m)  $\rho = 0.002$

Figure 10: Trade-off plot and grid of trajectory plots for various  $\rho$  values.

## A.2 Code and Output for Task 4

### Code

Listing 3: Python code for Task 4.

```
1 def optimize_vehicle_two_targets(A, B, E, x_init, q1, q2, T, p1, p2, rho,
2   plot_trajectory=True):
3     if T > q1.shape[1] or T > q2.shape[1]:
4         raise ValueError(f"T={T} is greater than the number of columns in q1 or
5           q2")
6     # CVXPY variables
7     x_var = cp.Variable((4, T))
8     u_var = cp.Variable((2, T-1))
9
10    # System constraints
11    constraints = [x_var[:, 0] == x_init]
12    for t in range(T-1):
13        constraints.append(x_var[:, t+1] == A @ x_var[:, t] + B @ u_var[:, t])
14
15    # Objective function
16    TE = sum(p1 * cp.norm(E @ x_var[:, t] - q1[:, t], 2) +
17            p2 * cp.norm(E @ x_var[:, t] - q2[:, t], 2) for t in range(T))
18    CE = cp.sum_squares(u_var)
19    cost = TE + rho * CE
20
21    # Solve optimization problem
22    prob = cp.Problem(cp.Minimize(cost), constraints)
23    prob.solve(solver=cp.SCS, verbose=False)
24
25    # Store metrics
26    TE_val = TE.value
27    CE_val = CE.value
28    x_traj = x_var.value.copy()
29
30    return x_traj, TE_val, CE_val
31
32 # Setting parameters
33 A = np.array([[1, 0, 0.1, 0],
34              [0, 1, 0, 0.1],
35              [0, 0, 0.8, 0],
36              [0, 0, 0, 0.8]])
37
38 B = np.array([[0, 0],
39              [0, 0],
40              [0.1, 0],
41              [0, 0.1]])
42
43 E = np.array([[1, 0, 0, 0],
44              [0, 1, 0, 0]])
45
46 x_init = np.array([0.5, 0, 1, -1])
47
48 # Load targets
49 q1 = np.load("target_1.npy")
50 q2 = np.load("target_2.npy")
51
52 # Instance parameters
53 T = 60
54 lista_p = [(0, 1), (0.1, 0.9), (0.2, 0.8), (0.3, 0.7), (0.4, 0.6), (0.5, 0.5), (0.6, 0.4),
55            (0.7, 0.3), (0.8, 0.2), (0.9, 0.1), (1, 0)]
```

```

56
57 rho = 0.1
58 for i, (p1, p2) in enumerate(lista_p, start=1):
59     x_traj, TE_val, CE_val = optimize_vehicle_two_targets(A, B, E, x_init, q1,
60         q2, T, p1, p2, rho, plot_trajectory=True)
61
62     print(f"instance {i}:\t ({TE_val:.2f}, {CE_val:.2f})")
63
64     # Plot trajectory
65     plt.figure(figsize=(6,6))
66     plt.plot(q1[0,:T], q1[1,:T], 'r-o', ms=4, lw=1, label='Target trajectory 1')
67     plt.plot(q2[0,:T], q2[1,:T], 'm-o', ms=4, lw=1, label='Target trajectory 2')
68     plt.plot(x_traj[0,:], x_traj[1,:], 'k-o', ms=4, lw=1, label='Vehicle
69         trajectory')
70     plt.xlabel('x')
71     plt.ylabel('y')
72     plt.title(f'Vehicle trajectory (p1={p1}, p2={p2}, rho={rho})')
73     plt.legend()
74     plt.minorticks_on()
75     plt.grid(which='major', linestyle='--', linewidth='0.8')
76     plt.grid(which='minor', linestyle=':', linewidth='0.5')
77     plt.xlim(-1.5, 1.5)
78     plt.ylim(-1.5, 1.5)
79     plt.savefig(f"export/task4/trajectory_instance{i}.pdf")
80     plt.show()

```

## Output

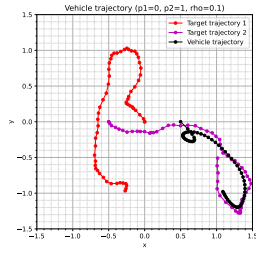
Listing 4: Console output for Task 4.

```

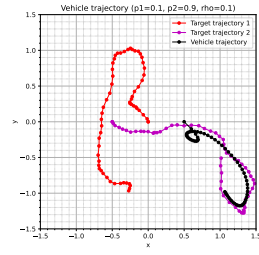
instance 1: (17.46, 69.61)
instance 2: (25.35, 67.46)
instance 3: (33.28, 63.74)
instance 4: (41.29, 56.37)
instance 5: (49.01, 43.77)
instance 6: (53.52, 38.18)
instance 7: (49.40, 63.24)
instance 8: (41.26, 99.82)
instance 9: (31.18, 137.90)
instance 10: (21.29, 164.06)
instance 11: (12.00, 179.38)

```

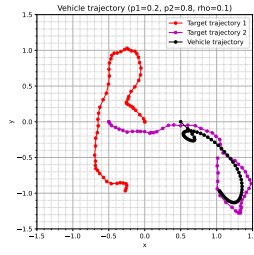
## Plots



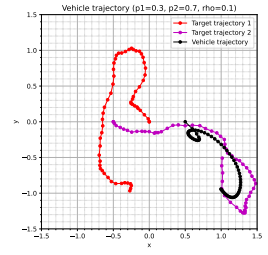
(a)  $\rho_1 = 0, \rho_2 = 1$



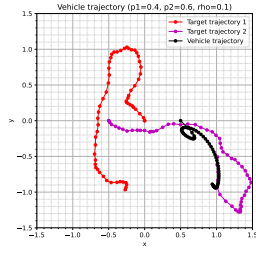
(b)  $\rho_1 = 0.1, \rho_2 = 0.9$



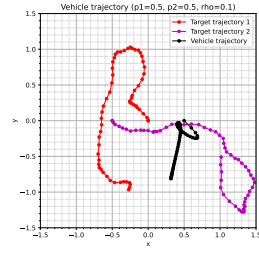
(c)  $\rho_1 = 0.2, \rho_2 = 0.8$



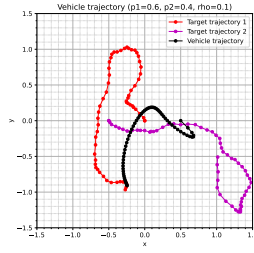
(d)  $\rho_1 = 0.3, \rho_2 = 0.7$



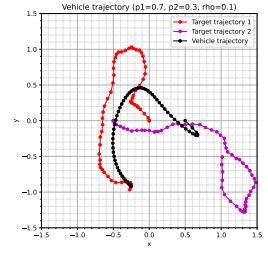
(e)  $\rho_1 = 0.4, \rho_2 = 0.6$



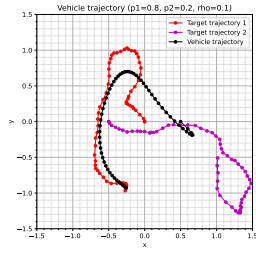
(f)  $\rho_1 = 0.5, \rho_2 = 0.5$



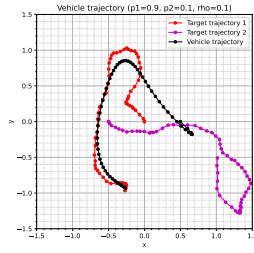
(g)  $\rho_1 = 0.6, \rho_2 = 0.4$



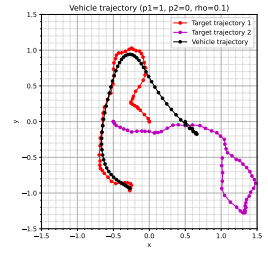
(h)  $\rho_1 = 0.7, \rho_2 = 0.3$



(i)  $\rho_1 = 0.8, \rho_2 = 0.2$



(j)  $\rho_1 = 0.9, \rho_2 = 0.1$



(k)  $\rho_1 = 1, \rho_2 = 0$

Figure 11: Grid of trajectory plots for various  $\rho_1$  and  $\rho_2$  values.

## A.3 Code and Output for Task 7

### Code

Listing 5: Python code for Task 7.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import cvxpy as cp
4 import os
5
6 def hybrid_trajectory_from_functions(A, B, E, x_init, q1, q2, T, tau, p1, p2,
7     rho,
8                                     plot_trajectory=True):
9
10     # Define all variables for both scenarios over the full horizon T
11     # Scenario 1 (target q1)
12     x1_var = cp.Variable((4, T))
13     u1_var = cp.Variable((2, T-1))
14     # Scenario 2 (target q2)
15     x2_var = cp.Variable((4, T))
16     u2_var = cp.Variable((2, T-1))
17
18     # Define constraints
19     constraints = [
20         # Initial state (shared)
21         x1_var[:, 0] == x_init,
22         x2_var[:, 0] == x_init,
23     ]
24
25     # Dynamics constraints (for both scenarios)
26     for t in range(T-1):
27         constraints.append(x1_var[:, t+1] == A @ x1_var[:, t] + B @ u1_var[:, t]
28                             ])
29         constraints.append(x2_var[:, t+1] == A @ x2_var[:, t] + B @ u2_var[:, t]
30                             ])
31
32     # Non-anticipativity constraints (from Task 6)
33     # The control signals u(1)...u(24) must be shared.
34     # In 0-based indexing for u_var[:, t], this is t = 0...23.
35     # tau = 25, so we loop from t=0 to tau-2.
36     for t in range(tau - 1): # t goes from 0 to 23
37         constraints.append(u1_var[:, t] == u2_var[:, t])
38     # This automatically enforces x1(t) == x2(t) for t=1...25
39
40     # Define the objective function (from Problem 5)
41     # Cost for scenario 1
42     TE1 = cp.sum(cp.norm(E @ x1_var - q1[:, :T], axis=0))
43     CE1 = cp.sum_squares(u1_var)
44     cost1 = TE1 + rho * CE1
45
46     # Cost for scenario 2
47     TE2 = cp.sum(cp.norm(E @ x2_var - q2[:, :T], axis=0))
48     CE2 = cp.sum_squares(u2_var)
49     cost2 = TE2 + rho * CE2
50
51     # Total expected cost
52     cost = p1 * cost1 + p2 * cost2
53
54     # Solve the problem
55     prob = cp.Problem(cp.Minimize(cost), constraints)
56     prob.solve(solver=cp.SCS, verbose=False)
57
58     # Extract results
```

```

56 x_traj_final_1 = x1_var.value
57 x_traj_final_2 = x2_var.value
58
59 TE_total_1 = TE1.value
60 CE_total_1 = CE1.value
61 TE_total_2 = TE2.value
62 CE_total_2 = CE2.value
63
64 # Split trajectories for return values (to match your main loop's needs)
65 x_traj1 = x_traj_final_1[:, :tau] # The shared part
66 x_traj2 = x_traj_final_1[:, tau:] # Branch 1
67 x_traj3 = x_traj_final_2[:, tau:] # Branch 2
68
69 # Plotting
70 if plot_trajectory:
71     plt.figure(figsize=(6,6))
72     plt.plot(q1[0,:], q1[1,:], 'r-o', lw=1, ms=4, label='Target trajectory 1
73             ')
74     plt.plot(q2[0,:], q2[1,:], 'm-o', lw=1, ms=4, label='Target trajectory 2
75             ')
76     plt.plot(x_traj_final_1[0,:], x_traj_final_1[1,:], 'k-o', lw=1, ms=4,
77             label='Vehicle if target follows 1')
78     plt.plot(x_traj_final_2[0,:], x_traj_final_2[1,:], 'g-o', lw=1, ms=4,
79             label='Vehicle if target follows 2')
80
81     # Highlight the split point (t=25, which is index 24)
82     plt.plot(x_traj_final_1[0, tau-1], x_traj_final_1[1, tau-1], 'bx', ms
83             =10, label='Pivot point')
84
85     plt.xlabel('x')
86     plt.ylabel('y')
87     plt.title(f'Hybrid trajectory (tau={tau}, p1={p1}, p2={p2}, rho={rho})')
88     plt.legend()
89     plt.minorticks_on()
90     plt.grid(which='major', linestyle='--', linewidth='0.8')
91     plt.grid(which='minor', linestyle=':', linewidth='0.5')
92     plt.xlim(-1.5, 1.5)
93     plt.ylim(-1.5, 1.5)
94     plt.savefig(f"export/task7/hybrid_p1_{p1}_p2_{p2}.pdf")
95     plt.show()
96
97 return (x_traj_final_1, x_traj_final_2,
98         TE_total_1, CE_total_1,
99         x_traj1, x_traj2, x_traj3,
100         TE_total_2, CE_total_2)
101
102 A = np.array([[1, 0, 0.1, 0],
103              [0, 1, 0, 0.1],
104              [0, 0, 0.8, 0],
105              [0, 0, 0, 0.8]])
106
107 B = np.array([[0, 0],
108              [0, 0],
109              [0.1, 0],
110              [0, 0.1]])
111
112 E = np.array([[1, 0, 0, 0],
113              [0, 1, 0, 0]])
114
115 x_init = np.array([0.5, 0, 1, -1])
116
117 T = 60
118 tau = 25

```

```

114
115 # lista_p = [(0.6, 0.4)]
116 # lista_p = [(0.4, 0.6), (0.9, 0.1)] # To check against examples
117 lista_p = [(0, 1), (0.1, 0.9), (0.2, 0.8), (0.3, 0.7), (0.4, 0.6), (0.5, 0.5), (0.6, 0.4),
118             (0.7, 0.3), (0.8, 0.2), (0.9, 0.1), (1, 0)] # Full list
119
120 rho = 0.1
121
122 q1 = np.load("target_1.npy") # shape: (2, N)
123 q2 = np.load("target_2.npy") # shape: (2, N)
124
125 for i, (p1, p2) in enumerate(lista_p, start=1):
126     (x_traj_final_1, x_traj_final_2,
127      TE_total_1, CE_total_1,
128      x_traj1, x_traj2, x_traj3,
129      TE_total_2, CE_total_2) = hybrid_trajectory_from_functions(
130          A=A, B=B, E=E, x_init=x_init,
131          q1=q1, q2=q2, T=T, tau=tau, p1=p1, p2=p2, rho=rho, plot_trajectory=True)
132
133     print(f"Instance {i} (p1={p1}, p2={p2}):")
134     print(f"    Scenario 1 (TE, CE): [{TE_total_1:.2f}, {CE_total_1:.2f}]")
135     print(f"    Scenario 2 (TE, CE): [{TE_total_2:.2f}, {CE_total_2:.2f}]")

```

## Output

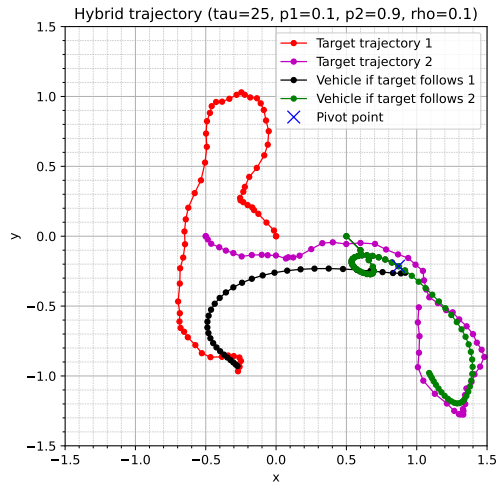
Listing 6: Console output for Task 9.

```

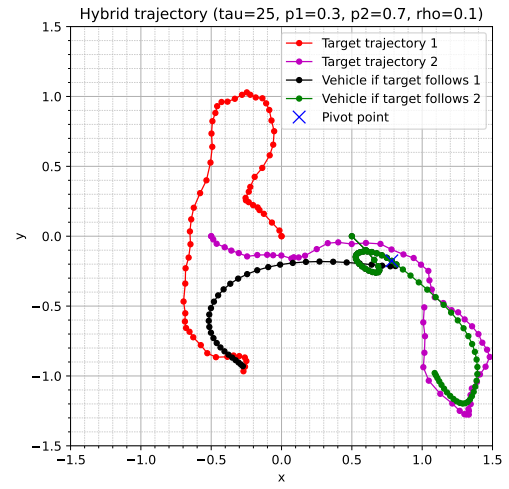
Instance 1 (p1=0, p2=1):
  Scenario 1 (TE, CE): [28.10, 52686.78]
  Scenario 2 (TE, CE): [17.46, 69.61]
Instance 2 (p1=0.1, p2=0.9):
  Scenario 1 (TE, CE): [49.07, 135.55]
  Scenario 2 (TE, CE): [17.54, 69.34]
Instance 3 (p1=0.2, p2=0.8):
  Scenario 1 (TE, CE): [48.19, 130.38]
  Scenario 2 (TE, CE): [17.62, 71.01]
Instance 4 (p1=0.3, p2=0.7):
  Scenario 1 (TE, CE): [46.42, 122.61]
  Scenario 2 (TE, CE): [18.04, 75.67]
Instance 5 (p1=0.4, p2=0.6):
  Scenario 1 (TE, CE): [42.85, 110.43]
  Scenario 2 (TE, CE): [19.65, 85.84]
Instance 6 (p1=0.5, p2=0.5):
  Scenario 1 (TE, CE): [38.33, 100.06]
  Scenario 2 (TE, CE): [22.28, 105.73]
Instance 7 (p1=0.6, p2=0.4):
  Scenario 1 (TE, CE): [32.84, 96.64]
  Scenario 2 (TE, CE): [26.82, 133.05]
Instance 8 (p1=0.7, p2=0.3):
  Scenario 1 (TE, CE): [25.07, 110.47]
  Scenario 2 (TE, CE): [34.48, 175.53]
Instance 9 (p1=0.8, p2=0.2):
  Scenario 1 (TE, CE): [18.46, 135.20]
  Scenario 2 (TE, CE): [42.25, 220.76]
Instance 10 (p1=0.9, p2=0.1):
  Scenario 1 (TE, CE): [14.06, 162.03]
  Scenario 2 (TE, CE): [47.97, 257.63]
Instance 11 (p1=1, p2=0):
  Scenario 1 (TE, CE): [12.00, 179.38]
  Scenario 2 (TE, CE): [24.89, 51941.43]

```

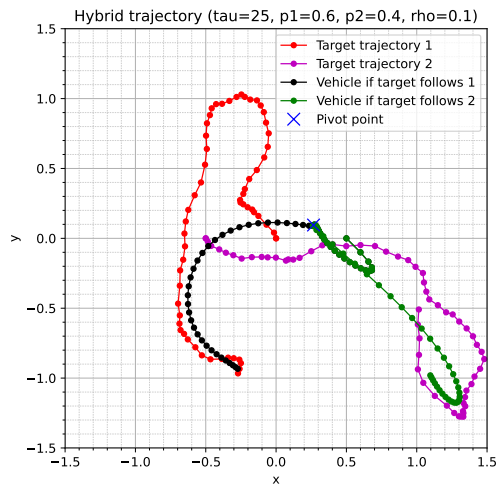
## Plots



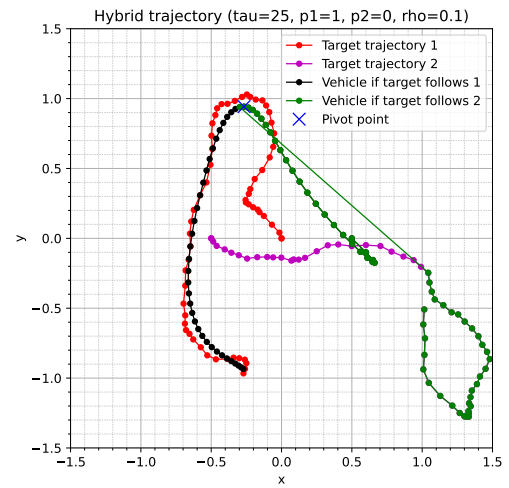
(a)  $p_1 = 0.1, p_2 = 0.9$



(b)  $p_1 = 0.3, p_2 = 0.7$



(c)  $p_1 = 0.6, p_2 = 0.4$



(d)  $p_1 = 1, p_2 = 0$

Figure 12: Grid of some outputted plots

## A.4 Code and Output for Task 9

### Code

Listing 7: Python code for Task 9.

```
1 # Load Data (noisy points)
2 dataset = np.load('circle_data_2.npy').T
3
4 # Get the number of data points, N
5 N = dataset.shape[0]
6
7 # Construct the A matrix and b vector
8 # A is an N x 3 matrix
9 A = np.zeros((N, 3))
10 # b is an N x 1 vector
11 b = np.zeros((N, 1))
12
13 # Iterate through each data point x_n to build the system
14 for i in range(N):
15     # Get the current point xn
16     xn = dataset[i, :]
17
18     # i-th row of matrix A: [1 -2*x_n^T]
19     A[i, :] = [1, -2 * xn[0], -2 * xn[1]]
20
21     # i-th element of vector b: -||x_n||^2
22     # Equivalent to -(x_n[0]**2 + x_n[1]**2)
23     b[i, 0] = -np.sum(xn**2)
24
25 # Solve the Least-Squares Problem: min ||Ax - b||^2
26 # The solution vector contains (y, c1, c2)
27 solution_ls, *_ = np.linalg.lstsq(A, b, rcond=None)
28
29 # Extract the Circle Parameters from the solution
30 y_ls = solution_ls[0, 0]
31 c_ls = solution_ls[1:, 0] # Store as a 1D array (2,)
32
33 # Calculate the radius using R = sqrt(||c||^2 - y)
34 R_ls = np.sqrt(np.linalg.norm(c_ls)**2 - y_ls)
35
36 # Print the Results
37 print("--- Results from LS Method ---")
38 print(f"Center c_ls: ({c_ls[0]:.2f}, {c_ls[1]:.2f})")
39 print(f"Radius R_ls: {R_ls:.2f}\n")
40
41 def plot_circle(ax, center, radius, **kwargs):
42     # Set default styles that match the PDF's appearance
43     style = {
44         'fill': False,
45         'linewidth': 2
46     }
47     # Allow to override defaults (e.g., color, linestyle, label)
48     style.update(kwargs)
49
50     circle_patch = Circle(xy=center, radius=radius, **style)
51     ax.add_patch(circle_patch)
52
53 fig, ax = plt.subplots(figsize=(8, 8))
54 ax.scatter(dataset[:, 0], dataset[:, 1], color='black', s=20, label='Noisy
    dataset')
55 plot_circle(ax, center=(-1, 1), radius=1.0, edgecolor='red', linestyle=':',
    label='Ground truth')
```

```

56 plot_circle(ax, center=c_ls, radius=R_ls, edgecolor='magenta', linestyle=':',
    label='LS fit')
57
58 ax.set_title("Task 9: LS Circle Fit on Dataset 2")
59 plt.minorticks_on()
60 plt.grid(which='major', linestyle='-', linewidth='0.8')
61 plt.grid(which='minor', linestyle=':', linewidth='0.5')
62 ax.legend()
63 ax.set_aspect('equal', adjustable='box')
64 plt.savefig(f"export/task9/LS_circle_fit.pdf")
65 plt.show()

```

## Output

Listing 8: Console output for Task 9.

```

--- Results from LS Method ---
Center c_ls: (-0.45, 1.50)
Radius R_ls: 0.59

```

## Plot

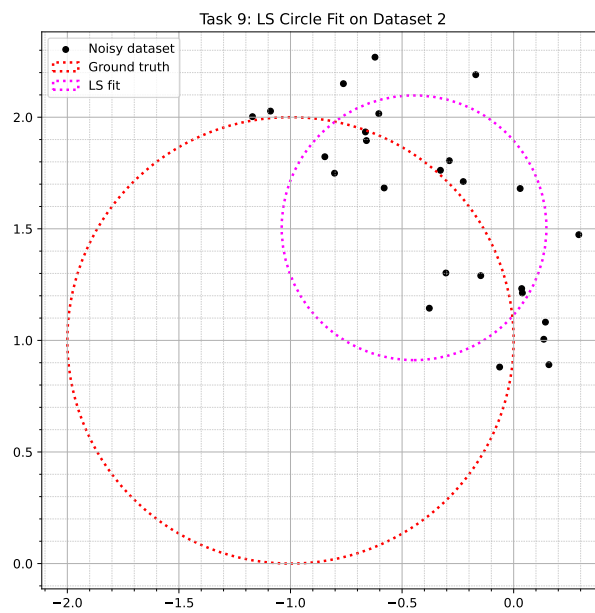


Figure 13: The ground truth circle (red) and the circle fitted by the LS approach (magenta).

## A.5 Code and Output for Task 10

### Code

Listing 9: Python code for Task 10.

```
1 # Load Data (noisy points)
2 xn = np.load('circle_data_2.npy').T
3
4 N = xn.shape[0]
5
6 # Cost function f(c, R) = sum(||c - x_n|| - R)^2
7 def cost_function(z, points):
8     c = z[:2]
9     R = z[2]
10    distances = np.linalg.norm(points - c, axis=1)
11    residuals = distances - R
12    return np.sum(residuals**2)
13
14 # Initialize LM parameters using the result from the LS method
15 # z_k is the parameter vector [c1, c2, R]
16 z_k = np.hstack([c_ls, R_ls])
17 lambda_k = 1.0    # Damping parameter
18 epsilon = 1e-6    # Stopping tolerance for the gradient norm
19 k = 0             # Iteration counter
20
21 # storage for gradient norms for plotting
22 gradient_norms = []
23
24 while True:
25     c_k = z_k[:2]
26     R_k = z_k[2]
27
28     # distance d = ||c_k - x_n||
29     distances = np.linalg.norm(xn - c_k, axis=1)
30     # residuals r = ||c_k - x_n|| - R_k = d - R_k
31     residuals = distances - R_k
32
33     # Calculate the Jacobian matrix J (size N x 3)
34     J = np.zeros((N, 3))
35     J[:, 0] = -(xn[:, 0] - c_k[0]) / distances
36     J[:, 1] = -(xn[:, 1] - c_k[1]) / distances
37     J[:, 2] = -1.0
38
39     # Gradient of the cost function g = 2 * J.T @ r
40     # g_k = f'(z_k)
41     # g_k = f'(z_k) = 2 * J(z_k)^T r(z_k)
42     gradient = 2 * J.T @ residuals
43     gradient_norm = np.linalg.norm(gradient)
44     gradient_norms.append(gradient_norm)
45
46     if gradient_norm < epsilon:
47         break
48
49     # Solve the LM system for the step delta_z
50     # (J.T @ J + lambda*I) @ delta_z = -J.T @ r
51     A_lm = np.vstack([J, np.sqrt(lambda_k) * np.identity(3)])
52     b_lm = np.hstack([-residuals, np.zeros(3)])
53     delta_z, *_ = np.linalg.lstsq(A_lm, b_lm, rcond=None)
54
55     # Propose a candidate for the next iterate
56     z_candidate = z_k + delta_z
57
58     # Evaluate if the step is good
```

```

59     cost_current = cost_function(z_k, xn)
60     cost_candidate = cost_function(z_candidate, xn)
61
62     # validation and update
63     if cost_candidate < cost_current:
64         z_k = z_candidate
65         lambda_k *= 0.7
66     else:
67         lambda_k *= 2
68
69     k = k+1
70
71 # Final optimal parameters
72 c_lm = z_k[:2]
73 R_lm = z_k[2]
74
75 print("--- Results from LM Method ---")
76 print(f"LM converged in {k} iterations.")
77 print(f"Final Center c_lm: ({c_lm[0]:.2f}, {c_lm[1]:.2f})")
78 print(f"Final Radius R_lm: {R_lm:.2f}")
79
80 def plot_circle(ax, center, radius, **kwargs):
81     # Set default styles that match the PDF's appearance
82     style = {
83         'fill': False,
84         'linewidth': 2
85     }
86     # Allow to override defaults (e.g., color, linestyle, label)
87     style.update(kwargs)
88
89     circle_patch = Circle(xy=center, radius=radius, **style)
90     ax.add_patch(circle_patch)
91
92 fig, ax = plt.subplots(figsize=(8, 8))
93 ax.scatter(xn[:, 0], xn[:, 1], color='black', s=20, label='Noisy dataset')
94 plot_circle(ax, center=(-1, 1), radius=1.0, edgecolor='red', linestyle=':',
95             label='Ground truth')
96 plot_circle(ax, center=c_ls, radius=R_ls, edgecolor='magenta', linestyle=':',
97             label='LS fit')
98 plot_circle(ax, center=c_lm, radius=R_lm, edgecolor='blue', linestyle=':', label
99             ='LM fit')
100
101 ax.set_title("Task 10: LS and LM Circle Fits on Dataset 2")
102 plt.minorticks_on()
103 plt.grid(which='major', linestyle='-', linewidth='0.8')
104 plt.grid(which='minor', linestyle=':', linewidth='0.5')
105 ax.legend()
106 ax.set_aspect('equal', adjustable='box')
107 plt.savefig(f"export/task10/LM_circle_fit.pdf")
108 plt.show()

```

## Output

Listing 10: Console output for Task 10.

```

--- Results from LM Method ---
LM converged in 35 iterations.
Final Center c_lm: (-0.88, 1.10)
Final Radius R_lm: 0.91

```

## Plot

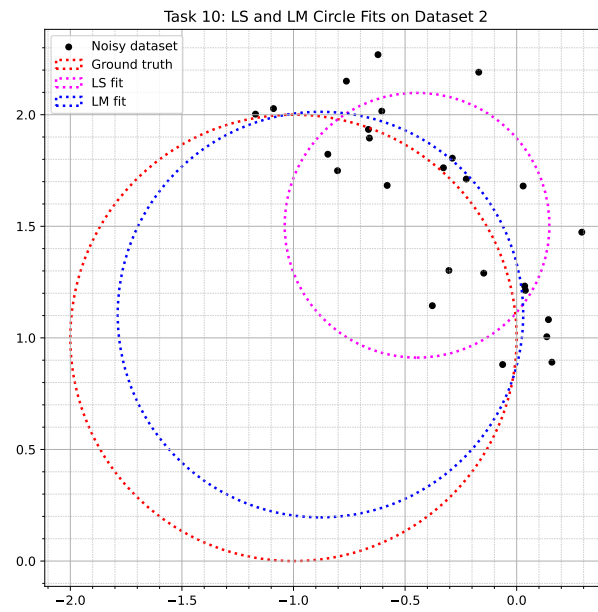


Figura 14: The ground truth circle (red), the circle fitted by the LS approach (magenta), and the circle fitted by the LM algorithm (blue).

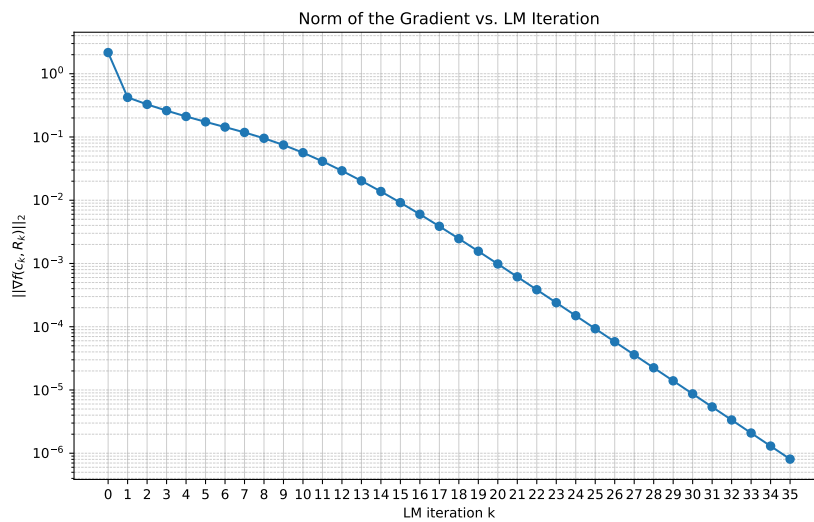


Figura 15: Norm of the gradient of the cost function  $f$  across iterations of the LM algorithm.

## A.6 Code and Output for Task 11

### Code

Listing 11: Python code for Task 11.

```
1 def build_Ab(dataset):
2     """Builds the A matrix and b vector for a given agent's dataset."""
3     N = dataset.shape[0]
4     A = np.zeros((N, 3))
5     b = np.zeros((N, 1)) # Use (N, 1) for consistent matrix operations
6     for n in range(N):
7         xn = dataset[n, :]
8         A[n, 0] = 1.0
9         A[n, 1] = -2 * xn[0]
10        A[n, 2] = -2 * xn[1]
11        b[n, 0] = -np.linalg.norm(xn)**2
12    return A, b
13
14 # Load the four datasets (transpose from 2xN to Nx2 format)
15 datasets = [np.load(f"dataset{i}.npz").T for i in range(1, 5)]
16 P = len(datasets) # Number of agents
17
18 local_circles = [] # To store (center, radius) for each agent
19 local_Ab = []      # To store (A, b) for each agent
20
21 for p in range(P):
22     Ap, bp = build_Ab(datasets[p])
23     local_Ab.append((Ap, bp))
24
25     # Solve the local LS problem: min ||Ap*x - bp||^2
26     # x_local is (y, c1, c2)
27     x_local, *_ = np.linalg.lstsq(Ap, bp, rcond=None)
28
29     y_local = x_local[0, 0]
30     c_local = x_local[1:, 0]
31     R_local = np.sqrt(np.linalg.norm(c_local)**2 - y_local)
32
33     local_circles.append({'center': c_local, 'radius': R_local})
34
35 # --- ALM Parameters ---
36 c = 100
37 epsilon_ALM = 0.01
38 epsilon_BCD = 0.01
39
40 # --- ALM Initialization ---
41 # lambda_p represents the list of multipliers {lambda_1, ..., lambda_P}, one for
42 # each agent
43 lambda_p = [np.zeros((3, 1)) for _ in range(P)]
44 # x0_k represents the server's variable x_0^k at the end of iteration k.
45 # Initialize x_0^0 = 0 as per the task.
46 x0_k = np.zeros((3, 1))
47
48 # --- BCD Pre-computation ---
49 LHS_precomputed = [] # Stores (2*Ap.T*Ap + c*I) for each agent
50 RHS_precomputed = [] # Stores (2*Ap.T*bp) for each agent
51 cI = c * np.identity(3)
52
53 for p in range(P):
54     Ap, bp = local_Ab[p]
55     LHS_precomputed.append(2 * Ap.T @ Ap + cI)
56     RHS_precomputed.append(2 * Ap.T @ bp)
57
58 # --- Tracking Lists for Reporting ---
```

```

58 alm_iterations = []
59 bcd_per_alm = []
60 max_normalized_violation_list = []
61
62 k = 0 # ALM iteration counter
63
64 while True:
65     k += 1
66     alm_iterations.append(k)
67
68     # --- Inner BCD Loop to minimize the Augmented Lagrangian ---
69     m = 0
70
71     # Initialize BCD with  $u_0^0 = x_0^{k-1}$ 
72     u0_m_minus_1 = np.copy(x0_k)
73
74     while True:
75         m += 1
76
77         # (c) Agent Update (in parallel) to find  $u_p^m$  for each agent p
78         #  $up_m = [np.linalg.solve(LHS\_precomputed[p], RHS\_precomputed[p] +$ 
79         #        $lambda\_p[p] + c * u0\_m\_minus\_1)$  for p in range(P)]
80
81         up_m = []
82
83         for p in range(P):
84             #  $RHS = 2*Ap.T*bp + lambda\_p^{k-1} + c*u0^{m-1}$ 
85             RHS = RHS_precomputed[p] + lambda_p[p] + c * u0_m_minus_1
86             LHS = LHS_precomputed[p]
87
88             # Solve  $(2*Ap.T*Ap + c*I) * up = RHS$ 
89             up = np.linalg.solve(LHS, RHS)
90             up_m.append(up)
91
92         # (e) Server Update to find  $u_0^m$ 
93         avg_up = np.mean(np.hstack(up_m), axis=1, keepdims=True)
94         avg_lambda = np.mean(np.hstack(lambda_p), axis=1, keepdims=True)
95         u0_m = avg_up - (1.0 / c) * avg_lambda
96
97         # (f) BCD Stop Condition
98         bcd_change = np.linalg.norm(u0_m - u0_m_minus_1)
99         if bcd_change <= epsilon_BCD:
100             break
101         else:
102             u0_m_minus_1 = u0_m
103
104     bcd_per_alm.append(m)
105
106     # Set results for ALM iteration k:  $(x_0^k, x_p^k) = (u_0^m, u_p^m)$ 
107     x0_k = u0_m
108     xp_k = up_m
109
110     # --- ALM: Update Multipliers and Check Stopping Condition ---
111     violations = []
112     for p in range(P):
113         violation_vec = x0_k - xp_k[p]
114         violations.append(np.linalg.norm(violation_vec))
115         # Update multiplier  $lambda\_p^k = lambda\_p^{k-1} + c * (x_0^k - x_p^k)$ 
116         lambda_p[p] = lambda_p[p] + c * violation_vec
117
118     # --- ALM: Check Stop Condition ---
119     max_unnormalized_violation = np.max(violations)
120     norm_x0_k = np.linalg.norm(x0_k)

```

```

120     max_normalized_violation = max_unnormalized_violation / norm_x0_k if
        norm_x0_k > 1e-9 else max_unnormalized_violation
121 source     max_normalized_violation_list.append(max_normalized_violation)
122
123     if max_normalized_violation <= epsilon_ALM:
124         break
125
126 x_FL = x0_k
127 y_FL = x_FL[0, 0]
128 c_FL = x_FL[1:, 0]
129 R_FL = np.sqrt(np.linalg.norm(c_FL)**2 - y_FL)
130
131 print("\n--- Final Federated Learning Results ---")
132 print(f"Center c_FL: ({c_FL[0]:.2f}, {c_FL[1]:.2f})")
133 print(f"Radius R_FL: {R_FL:.2f}")
134 print(f"\nTotal ALM iterations: {k}")
135 print(f"BCD iterations per ALM iteration: {bcd_per_alm}")
136
137 def plot_circle(ax, center, radius, **kwargs):
138     # Set default styles that match the PDF's appearance
139     style = {
140         'fill': False,
141         'linewidth': 2
142     }
143     # Allow to override defaults (e.g., color, linestyle, label)
144     style.update(kwargs)
145
146     circle_patch = Circle(xy=center, radius=radius, **style)
147     ax.add_patch(circle_patch)
148
149 fig, ax = plt.subplots(figsize=(10, 8))
150 agent_colors = ['#1f77b4', '#aec7e8', '#ff7f0e', '#ffbb78']
151 for i in range(P):
152     ax.scatter(datasets[i][:, 0], datasets[i][:, 1], color=agent_colors[i],
153               label=f"Dataset of agent {i+1}", alpha=0.7)
154     plot_circle(ax, local_circles[i]['center'], local_circles[i]['radius'],
155               edgecolor=agent_colors[i], linestyle='--', label=f"Circle fitted by agent
156               {i+1}", alpha=0.8)
157
158 plot_circle(ax, c_FL, R_FL, edgecolor='black', linestyle=':', label="Circle
159 fitted by FL", linewidth=3)
160 plot_circle(ax, (1, 5), 4, edgecolor='red', linestyle=':', label='Ground truth (
161 approx.)', linewidth=2)
162
163 ax.set_title(f"Task 11: Federated Learning Circle Fit (c={c})")
164 ax.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
165 plt.minorticks_on()
166 plt.grid(which='major', linestyle='--', linewidth='0.8')
167 plt.grid(which='minor', linestyle=':', linewidth='0.5')
168 ax.set_aspect('equal', adjustable='box')
169 plt.savefig(f"export/task11/Federated_Learning_circle_fit.pdf")
170 plt.show()
171
172 fig, ax = plt.subplots(figsize=(10, 6))
173 ax.semilogy(alm_iterations, max_normalized_violation_list, marker='o', linestyle
174 = '--')
175 ax.set_title(f"Maximum Constraint Violation per ALM Iteration (c={c})")
176 ax.set_xlabel("ALM iteration k")
177 ax.set_ylabel(r"$\max_{\mathbf{p}} \left\{ \frac{\|x_0^k - x_p^k\|_2}{\|x_0^k\|_2} \right\}$")
178
179 ax.grid(True, which='major', axis='x', linestyle='--', linewidth=0.5, color='gray',
180       alpha=0.4)
181 ax.grid(True, which='both', axis='y', linestyle='--', linewidth=0.5, color='gray'

```

```

174     ', alpha=0.4)
175 ax.set_xticks(alm_iterations)
176 plt.savefig(f"export/task11/ALM_constraint_violation.pdf")
177 plt.show()

```

## Output

Listing 12: Console output for Task 11.

```

--- Final Federated Learning Results ---
Center c_FL: (1.22, 5.05)
Radius R_FL: 4.08

Total ALM iterations: 4
BCD iterations per ALM iteration: [170, 3, 3, 2]

```

## Plot

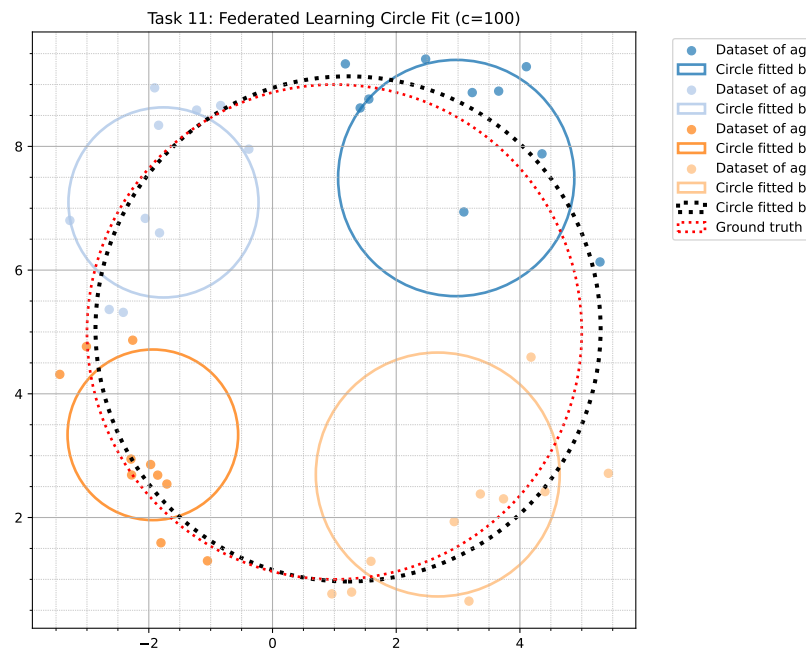


Figura 16: The four local datasets and the corresponding four local circles fitted by the four agents on their own, plus the circle fitted by Federated Learning.

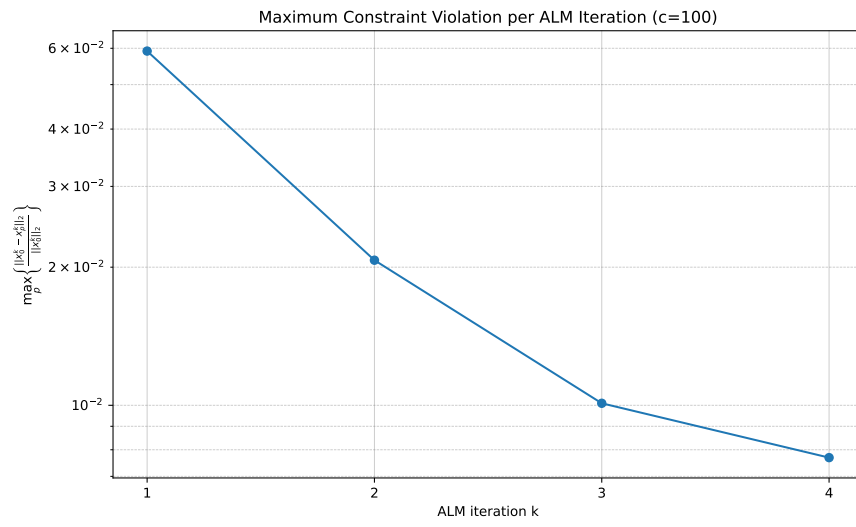


Figura 17: Maximum constraint violation across ALM iterations, for the case  $c = 100$ .